Vellore Institute of Technology

Vehicle Rental Management System

Course Project – Review II Report

S.No	Team member	Register number
1	**P. Sarath Chandra**	24BYB0047
2	**Palli Pradeep**	24BYB0025
3	**M. Prasanna**	24BDE0078

Item	Details
Course	BCSE302L – Database Systems
Milestone	Review II – Database Implementation
Slot	L1 + L2
Faculty	Dr. Ram Kumar T
Database	Oracle Database, SQL*Plus (Run SQL Command Line)

Contents

Section	Title	Rubric marks
0	Implementation summary and link to Review I	–
1	DDL and constraints	2
2	DML and data population	1
3	SQL queries	2
4	Procedures and functions	2
5	Cursors and triggers	2
6	Real-world transaction flow and business rules	–
7	Viva preparation: SQL / PL/SQL understanding	1
8	Conclusion	–

0 Implementation Summary

In Review I we designed the database for a multi-branch vehicle rental business: eight entities, ten one-to-many relationships, twelve functional requirements (FR-01 to FR-12) and eight business rules (BR-01 to BR-08). In this review the same design is implemented in Oracle. Table names, column names, data types and constraints are exactly those listed in Section 4.3 of the Review I report; no table or column was added or removed. The business rules are enforced inside the database through constraints, functions, procedures and triggers.

Rubric component	Section	What is implemented
DDL & Constraints (2)	1	8 tables, 4 sequences; 8 PRIMARY KEY, 10 FOREIGN KEY, 10 UNIQUE, 23 CHECK, 34 NOT NULL constraints and 13 DEFAULT values. 8 constraint tests.
DML & Data Population (1)	2	140 rows (10 to 45 per table). INSERT with DEFAULT, UPDATE, and DELETE with ON DELETE CASCADE.
SQL Queries (2)	3	16 queries: inner/outer/self joins, GROUP BY, HAVING, nested and correlated subqueries, MINUS, UNION ALL, CASE, date functions, top-N.
Procedures & Functions (2)	4	6 functions (age, rental days, rent, late fee, availability, balance) and 5 procedures (register, book, pickup, return, cancel).
Cursor & Triggers (2)	5	2 cursor procedures (explicit cursor and cursor FOR loop) and 4 row-level triggers.
Viva / Understanding (1)	6, 7	Transaction flow table, business-rule matrix and likely viva questions.

Schema from Review I

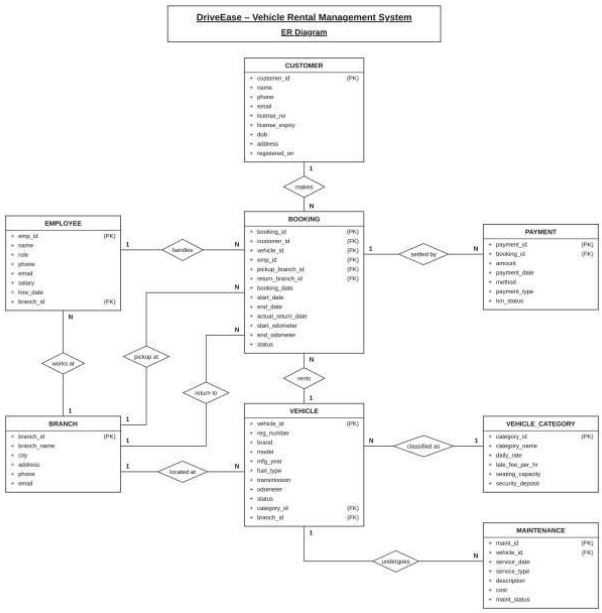


Figure 1: *ER diagram of the Vehicle Rental Management System (same as Review I)*

```
BRANCH (branch_id, branch_name, city, address, phone, email)
VEHICLE_CATEGORY (category_id, category_name, daily_rate, late_fee_per_hr, seating_capacity,
security_deposit)
VEHICLE (vehicle_id, reg_number, brand, model, mfg_year, fuel_type, transmission, odometer, status,
         category_id -> VEHICLE_CATEGORY, branch_id -> BRANCH)
CUSTOMER (customer_id, name, phone, email, license_no, license_expiry, dob, address, registered_on)
EMPLOYEE (emp_id, name, role, phone, email, salary, hire_date, branch_id -> BRANCH)
BOOKING (booking_id, customer_id -> CUSTOMER, vehicle_id -> VEHICLE, emp_id -> EMPLOYEE,
         pickup_branch_id -> BRANCH, return_branch_id -> BRANCH, booking_date, start_date, end_date,
         actual_return_date, start_odometer, end_odometer, status)
PAYMENT (payment_id, booking_id -> BOOKING, amount, payment_date, method, payment_type, txn_status)
MAINTENANCE (maint_id, vehicle_id -> VEHICLE, service_date, service_type, description, cost,
maint_status)
```

Order of execution

Step	Script (folder sql/)	What it does	Figures
0	00_setup_display.sql	Display settings (line size, column widths, date format). Run once per session.	–
1	01_ddl.sql	Creates the 4 sequences and 8 tables (parent tables first).	2
2	02_insert_data.sql	Loads the 140 sample rows and sets current vehicle states; COMMIT.	–
3	03_check_schema_data.sql	DESC, constraint summary, row counts and table listings (read only).	3–5, 8–10
4	04_constraint_tests.sql	Eight statements that must fail; ends with ROLLBACK.	6–7
5	05_queries.sql	Queries Q1–Q16 on the loaded data.	14–29
6	06_dml_demo.sql	INSERT, UPDATE and DELETE demonstration (changes a few rows).	11–13
7	07 to 10 (.sql)	Functions, procedures, cursors and triggers (CREATE OR REPLACE).	–
8	11_transactions.sql	Runs the PL/SQL objects block by block, in order.	30–43

99_cleanup.sql drops every object so the whole sequence can be repeated on fresh data. Steps 5 and 6 must stay in this order because the DML demonstration changes rows that the queries read.

1 DDL and Constraints

The tables are created parent first (BRANCH and VEHICLE_CATEGORY, then VEHICLE, CUSTOMER and EMPLOYEE, then BOOKING, and finally PAYMENT and MAINTENANCE) so that every foreign key points to a table that already exists. Every constraint is given a name with a prefix (`pk_`, `fk_`, `uq_`, `ck_`). Because of this, an error message such as ORA-02290 directly shows which rule was broken. The four sequences are used only by the procedures; they start after the IDs used in the sample data.

1.1 Sequences and table definitions

```
-- Sequences generate IDs inside the procedures. They start after the sample data.
CREATE SEQUENCE seq_customer    START WITH 13 INCREMENT BY 1 NOCACHE;
CREATE SEQUENCE seq_booking     START WITH 23 INCREMENT BY 1 NOCACHE;
CREATE SEQUENCE seq_payment     START WITH 46 INCREMENT BY 1 NOCACHE;
CREATE SEQUENCE seq_maintenance START WITH 13 INCREMENT BY 1 NOCACHE;
```

```
CREATE TABLE branch (
  branch_id    NUMBER(4)     CONSTRAINT pk_branch PRIMARY KEY,
  branch_name  VARCHAR2(50)  NOT NULL CONSTRAINT uq_branch_name UNIQUE,
  city         VARCHAR2(30)  NOT NULL,
  address      VARCHAR2(120) NOT NULL,
  phone        VARCHAR2(10)  NOT NULL CONSTRAINT uq_branch_phone UNIQUE
                             CONSTRAINT ck_branch_phone CHECK (LENGTH(phone) = 10),
  email        VARCHAR2(60)  CONSTRAINT uq_branch_email UNIQUE
);

CREATE TABLE vehicle_category (
  category_id      NUMBER(3)    CONSTRAINT pk_category PRIMARY KEY,
  category_name    VARCHAR2(30) NOT NULL CONSTRAINT uq_category_name UNIQUE,
  daily_rate       NUMBER(8,2)  NOT NULL CONSTRAINT ck_category_rate CHECK (daily_rate > 0),
  late_fee_per_hr  NUMBER(7,2)  DEFAULT 100  CONSTRAINT ck_category_latefee CHECK (late_fee_per_hr >= 0),
  seating_capacity NUMBER(2)    CONSTRAINT ck_category_seats CHECK (seating_capacity BETWEEN 1 AND 15),
  security_deposit NUMBER(8,2)  DEFAULT 2000 CONSTRAINT ck_category_deposit CHECK (security_deposit >= 0)
);

CREATE TABLE vehicle (
  vehicle_id   NUMBER(5)    CONSTRAINT pk_vehicle PRIMARY KEY,
  reg_number   VARCHAR2(12) NOT NULL CONSTRAINT uq_vehicle_reg UNIQUE,
  brand        VARCHAR2(30) NOT NULL,
  model        VARCHAR2(30) NOT NULL,
  mfg_year     NUMBER(4)    CONSTRAINT ck_vehicle_year CHECK (mfg_year >= 2010),
  fuel_type    VARCHAR2(10) NOT NULL
               CONSTRAINT ck_vehicle_fuel CHECK (fuel_type IN ('PETROL', 'DIESEL', 'CNG', 'ELECTRIC')),
  transmission VARCHAR2(10) DEFAULT 'MANUAL'
               CONSTRAINT ck_vehicle_trans CHECK (transmission IN ('MANUAL', 'AUTOMATIC')),
  odometer     NUMBER(7)    DEFAULT 0 CONSTRAINT ck_vehicle_odometer CHECK (odometer >= 0),
  status       VARCHAR2(12) DEFAULT 'AVAILABLE'
               CONSTRAINT ck_vehicle_status
               CHECK (status IN ('AVAILABLE', 'RENTED', 'MAINTENANCE', 'RETIRED')),
  category_id  NUMBER(3)    NOT NULL
               CONSTRAINT fk_vehicle_category REFERENCES vehicle_category (category_id),
  branch_id    NUMBER(4)    NOT NULL CONSTRAINT fk_vehicle_branch REFERENCES branch (branch_id)
```

```
);
```

```
CREATE TABLE customer (
  customer_id    NUMBER(6)     CONSTRAINT pk_customer PRIMARY KEY,
  name           VARCHAR2(50)  NOT NULL,
  phone          VARCHAR2(10)  NOT NULL CONSTRAINT uq_customer_phone UNIQUE,
  email          VARCHAR2(60)  CONSTRAINT uq_customer_email UNIQUE,
  license_no     VARCHAR2(16)  NOT NULL CONSTRAINT uq_customer_license UNIQUE,
  license_expiry DATE          NOT NULL,
  dob            DATE          NOT NULL,
  address        VARCHAR2(120),
  registered_on  DATE          DEFAULT SYSDATE
);
-- Age >= 21 and licence validity use SYSDATE, so they are checked by a trigger (Section 5).

CREATE TABLE employee (
  emp_id     NUMBER(5)    CONSTRAINT pk_employee PRIMARY KEY,
  name       VARCHAR2(50) NOT NULL,
  role       VARCHAR2(15) NOT NULL
             CONSTRAINT ck_employee_role CHECK (role IN ('MANAGER', 'AGENT', 'MECHANIC', 'CASHIER')),
  phone      VARCHAR2(10) NOT NULL CONSTRAINT uq_employee_phone UNIQUE,
  email      VARCHAR2(60) CONSTRAINT uq_employee_email UNIQUE,
  salary     NUMBER(9,2)  CONSTRAINT ck_employee_salary CHECK (salary > 0),
  hire_date  DATE         DEFAULT SYSDATE,
  branch_id  NUMBER(4)    NOT NULL CONSTRAINT fk_employee_branch REFERENCES branch (branch_id)
);
```

```
CREATE TABLE booking (
  booking_id         NUMBER(7) CONSTRAINT pk_booking PRIMARY KEY,
  customer_id        NUMBER(6) NOT NULL CONSTRAINT fk_booking_customer REFERENCES customer (customer_id),
  vehicle_id         NUMBER(5) NOT NULL CONSTRAINT fk_booking_vehicle  REFERENCES vehicle (vehicle_id),
  emp_id             NUMBER(5) NOT NULL CONSTRAINT fk_booking_employee REFERENCES employee (emp_id),
  pickup_branch_id   NUMBER(4) NOT NULL CONSTRAINT fk_booking_pickup   REFERENCES branch (branch_id),
  return_branch_id   NUMBER(4) NOT NULL CONSTRAINT fk_booking_return   REFERENCES branch (branch_id),
  booking_date       DATE      DEFAULT SYSDATE,
  start_date         DATE      NOT NULL,
  end_date           DATE      NOT NULL,
  actual_return_date DATE,
  start_odometer     NUMBER(7),
  end_odometer       NUMBER(7),
  status             VARCHAR2(12) DEFAULT 'BOOKED'
                     CONSTRAINT ck_booking_status
                     CHECK (status IN ('BOOKED', 'ONGOING', 'COMPLETED', 'CANCELLED')),
  CONSTRAINT ck_booking_dates    CHECK (end_date > start_date),
  CONSTRAINT ck_booking_return   CHECK (actual_return_date >= start_date),
  CONSTRAINT ck_booking_odometer CHECK (end_odometer >= start_odometer)
);
```

```
CREATE TABLE payment (
  payment_id   NUMBER(8)    CONSTRAINT pk_payment PRIMARY KEY,
  booking_id   NUMBER(7)    NOT NULL CONSTRAINT fk_payment_booking
                            REFERENCES booking (booking_id) ON DELETE CASCADE,
  amount       NUMBER(9,2)  NOT NULL CONSTRAINT ck_payment_amount CHECK (amount > 0),
  payment_date DATE         DEFAULT SYSDATE,
  method       VARCHAR2(12) NOT NULL
               CONSTRAINT ck_payment_method CHECK (method IN ('CASH', 'CARD', 'UPI', 'NETBANKING')),
  payment_type VARCHAR2(10) NOT NULL
               CONSTRAINT ck_payment_type CHECK (payment_type IN ('ADVANCE', 'FINAL', 'LATE_FEE', 'REFUND')),
  txn_status   VARCHAR2(10) DEFAULT 'SUCCESS'
               CONSTRAINT ck_payment_status CHECK (txn_status IN ('SUCCESS', 'PENDING', 'FAILED'))
);

CREATE TABLE maintenance (
  maint_id     NUMBER(6)     CONSTRAINT pk_maintenance PRIMARY KEY,
  vehicle_id   NUMBER(5)     NOT NULL CONSTRAINT fk_maintenance_vehicle REFERENCES vehicle (vehicle_id),
  service_date DATE          DEFAULT SYSDATE,
  service_type VARCHAR2(20)  NOT NULL
               CONSTRAINT ck_maint_type
               CHECK (service_type IN ('GENERAL', 'REPAIR', 'TYRE', 'ACCIDENT', 'INSURANCE')),
  description  VARCHAR2(200),
  cost         NUMBER(9,2)   DEFAULT 0 CONSTRAINT ck_maint_cost CHECK (cost >= 0),
  maint_status VARCHAR2(12)  DEFAULT 'SCHEDULED'
               CONSTRAINT ck_maint_status CHECK (maint_status IN ('SCHEDULED', 'IN_PROGRESS', 'DONE'))
);
```

SCREENSHOT PLACEHOLDER – Figure 2

Delete this box and paste the screenshot (@00_setup_display.sql then @01_ddl.sql). Listed as Figure 2 in the screenshot checklist.

Figure 2: *Running 01_ddl.sql: four sequences and eight tables created*

SCREENSHOT PLACEHOLDER – Figure 3

Delete this box and paste the screenshot (run block SC_DESC of 03_check_schema_data.sql). Listed as Figure 3 in the screenshot checklist.

Figure 3: *Structure of the BOOKING and PAYMENT tables (DESC)*

DESC confirms that the columns, data types and NOT NULL settings match the Review I design. The primary key column is also shown as NOT NULL.

1.2 Constraints used

Constraint	Count	Where it is used	Purpose
PRIMARY KEY	8	One per table: branch_id, category_id, vehicle_id, customer_id, emp_id, booking_id, payment_id, maint_id	Identifies every row uniquely.
FOREIGN KEY	10	vehicle (2), employee (1), booking (5), payment (1), maintenance (1)	One per relationship in the ER diagram. PAYMENT uses ON DELETE CASCADE, as planned in Review I.
UNIQUE	10	branch name/phone/email, category name, reg_number, customer phone/email/licence, employee phone/email	Stops duplicate vehicles, customers and contact details.
CHECK	23	Allowed values of status, fuel, role, method, payment type; rate > 0; seats 1–15; mfg_year >= 2010; phone length 10; end_date > start_date; end_odometer >= start_odometer (BR-08)	Keeps values inside the business limits.
NOT NULL	34	Names, phones, dates and every foreign key on the "many" side	Total participation from the ER diagram: e.g. a vehicle cannot exist without a category and a branch.
DEFAULT	13	status 'AVAILABLE'/'BOOKED'/'SCHEDULED', transmission 'MANUAL', odometer 0, late fee 100, deposit 2000, txn_status 'SUCCESS', cost 0, dates SYSDATE	Sensible value when a column is left out.

Age ≥ 21 and licence validity depend on the current date. Oracle does not allow SYSDATE inside a CHECK constraint, so (as noted in Review I) these two rules are enforced by triggers in Section 5.

SCREENSHOT PLACEHOLDER – Figure 4

Delete this box and paste the screenshot (run block SC_COUNT of 03_check_schema_data.sql). Listed as Figure 4 in the screenshot checklist.

Figure 4: *Number of constraints of each kind, read from USER_CONSTRAINTS*

SCREENSHOT PLACEHOLDER – Figure 5

Delete this box and paste the screenshot (run block SC_FK of 03_check_schema_data.sql). Listed as Figure 5 in the screenshot checklist.

Figure 5: *The ten foreign keys and the table each one refers to*

1.3 Constraint testing (each statement must fail)

#	Statement (04_constraint_tests.sql)	Constraint	Expected Oracle error
1	Insert vehicle with reg_number TN09AB1001 (already used)	UNIQUE	ORA-00001 UQ_VEHICLE_REG
2	Insert vehicle with fuel_type 'HYBRID'	CHECK	ORA-02290 CK_VEHICLE_FUEL
3	Insert branch with phone '98401' (5 digits)	CHECK	ORA-02290 CK_BRANCH_PHONE
4	Insert customer without license_no	NOT NULL	ORA-01400 CUSTOMER.LICENSE_NO
5	Insert payment for booking 999	FOREIGN KEY (child)	ORA-02291 FK_PAYMENT_BOOKING
6	Delete customer 1, who has bookings	FOREIGN KEY (parent)	ORA-02292 FK_BOOKING_CUSTOMER
7	Set end_date one day before start_date	Table CHECK	ORA-02290 CK_BOOKING_DATES
8	Set end_odometer below start_odometer (BR-08)	Table CHECK	ORA-02290 CK_BOOKING_ODOMETER

SCREENSHOT PLACEHOLDER – Figure 6

Delete this box and paste the screenshot (run block CT1 of 04_constraint_tests.sql). Listed as Figure 6 in the screenshot checklist.

Figure 6: *Constraint tests 1-4: UNIQUE, CHECK and NOT NULL violations rejected*

SCREENSHOT PLACEHOLDER – Figure 7

Delete this box and paste the screenshot (run block CT2 of 04_constraint_tests.sql). Listed as Figure 7 in the screenshot checklist.

Figure 7: *Constraint tests 5-8: foreign key and table-level CHECK violations rejected*

2 DML and Data Population

2.1 Sample data (02_insert_data.sql)

The sample data describes a fictional chain, "DriveEase", with ten branches in six cities. It was written so that the data itself follows the business rules, not just the constraints:

- Each booking starts at the branch where the vehicle was parked at that time, and is handled by the agent of that branch.
- Odometer readings follow on from one rental to the next; a vehicle's current odometer equals the end reading of its last rental.
- Every FINAL payment equals rent minus advance, and every LATE_FEE equals hours late × the category rate (BR-04, BR-05).
- The two cancellations follow BR-06: booking 21 was cancelled 2.7 days early (full refund), booking 22 five hours before pickup (50%).
- The data covers all states: 16 completed, 2 ongoing, 2 booked and 2 cancelled bookings; vehicles AVAILABLE, RENTED, MAINTENANCE and RETIRED; payments SUCCESS, PENDING and FAILED; three one-way rentals (BR-07).

Table	Rows	Table	Rows
BRANCH	10	EMPLOYEE	15
VEHICLE_CATEGORY	10	BOOKING	22
VEHICLE	14	PAYMENT	45
CUSTOMER	12	MAINTENANCE	12

Representative INSERT statements are shown below; the full script has one INSERT per row.

```
INSERT INTO branch VALUES (1, 'DriveEase Chennai Central', 'Chennai', '12 Anna Salai, Chennai', '9840100001',
'chennai.central@driveease.in');
INSERT INTO branch VALUES (2, 'DriveEase Chennai Airport', 'Chennai', 'GST Road, Meenambakkam, Chennai',
'9840100002', 'chennai.airport@driveease.in');

INSERT INTO vehicle_category VALUES (1, 'Hatchback', 1500, 150, 5, 2000);
INSERT INTO vehicle_category VALUES (2, 'Sedan', 2200, 200, 5, 3000);
INSERT INTO vehicle_category VALUES (3, 'Compact SUV', 2800, 250, 5, 4000);
INSERT INTO vehicle_category VALUES (4, 'SUV', 3500, 300, 7, 5000);
INSERT INTO vehicle_category VALUES (5, 'MUV', 3200, 300, 7, 5000);
INSERT INTO vehicle_category VALUES (6, 'Luxury Sedan', 6000, 500, 5, 10000);
INSERT INTO vehicle_category VALUES (7, 'Electric Car', 3000, 250, 5, 5000);
INSERT INTO vehicle_category VALUES (8, 'Tempo Traveller', 5500, 450, 12, 8000);
INSERT INTO vehicle_category VALUES (9, 'Scooter', 500, 50, 2, 1000);
-- Motorbike: late_fee_per_hr and security_deposit are left out, so the DEFAULT values (100, 2000) are used
INSERT INTO vehicle_category (category_id, category_name, daily_rate, seating_capacity)
  VALUES (10, 'Motorbike', 800, 2);

INSERT INTO vehicle VALUES (1, 'TN09AB1001', 'Maruti Suzuki', 'Swift', 2022, 'PETROL', 'MANUAL', 28530,
'AVAILABLE', 1, 1);
INSERT INTO vehicle VALUES (2, 'TN09AB1002', 'Hyundai', 'Creta', 2023, 'DIESEL', 'AUTOMATIC', 15350,
'AVAILABLE', 3, 3);

INSERT INTO customer VALUES (1, 'Aditya Raman', '9841000101', 'aditya.raman@gmail.com', 'TN0920180012345', DATE
'2038-03-13', DATE '1996-03-14', 'Adyar, Chennai', DATE '2026-01-12');
```

```
INSERT INTO customer VALUES (2, 'Meera Krishnan', '9841000102', 'meera.k@gmail.com', 'TN0120120034567', DATE
'2032-07-21', DATE '1990-07-22', 'T Nagar, Chennai', DATE '2026-02-03');

-- Columns: booking_id, customer_id, vehicle_id, emp_id, pickup_branch_id, return_branch_id,
--          booking_date, start_date, end_date, actual_return_date, start_odometer, end_odometer, status
INSERT INTO booking VALUES (2, 2, 2, 2, 1, 3, DATE '2026-08-01',
  TO_DATE('2026-08-05 10:00', 'YYYY-MM-DD HH24:MI'), TO_DATE('2026-08-08 10:00', 'YYYY-MM-DD HH24:MI'),
  TO_DATE('2026-08-08 14:00', 'YYYY-MM-DD HH24:MI'), 15000, 15350, 'COMPLETED');

-- payments of booking 2 (advance, final, late fee)
INSERT INTO payment VALUES (2, 2, 3000, TO_DATE('2026-08-01 11:00', 'YYYY-MM-DD HH24:MI'), 'CARD', 'ADVANCE',
'SUCCESS');
INSERT INTO payment VALUES (5, 2, 5400, TO_DATE('2026-08-08 14:00', 'YYYY-MM-DD HH24:MI'), 'CARD', 'FINAL',
'SUCCESS');
INSERT INTO payment VALUES (6, 2, 1000, TO_DATE('2026-08-08 14:01', 'YYYY-MM-DD HH24:MI'), 'CARD', 'LATE_FEE',
'SUCCESS');

-- Current vehicle states: two vehicles are out on rent, one is in the workshop, one is retired
UPDATE vehicle SET status = 'RENTED'      WHERE vehicle_id IN (2, 5);
UPDATE vehicle SET status = 'MAINTENANCE' WHERE vehicle_id IN (8);
UPDATE vehicle SET status = 'RETIRED'     WHERE vehicle_id IN (14);

COMMIT;
```

SCREENSHOT PLACEHOLDER – Figure 8

Delete this box and paste the screenshot (run block DATA_COUNTS of 03_check_schema_data.sql). Listed as Figure 8 in the screenshot checklist.

Figure 8: *Row count of every table after loading 02_insert_data.sql*

SCREENSHOT PLACEHOLDER – Figure 9

Delete this box and paste the screenshot (run block DATA_VEHICLE of 03_check_schema_data.sql). Listed as Figure 9 in the screenshot checklist.

Figure 9: *VEHICLE table after loading: current odometer, status and branch of each vehicle*

SCREENSHOT PLACEHOLDER – Figure 10

Delete this box and paste the screenshot (run block DATA_BOOKING of 03_check_schema_data.sql). Listed as Figure 10 in the screenshot checklist.

Figure 10: *BOOKING table after loading: 16 completed, 2 ongoing, 2 booked and 2 cancelled rentals*

2.2 INSERT, UPDATE and DELETE operations (06_dml_demo.sql)

These statements are run after the queries of Section 3 because they change a few rows.

```
-- A new Kia Seltos joins the Chennai Airport branch. transmission, odometer and status
-- are not given, so the DEFAULT values ('MANUAL', 0, 'AVAILABLE') are stored.
INSERT INTO vehicle (vehicle_id, reg_number, brand, model, mfg_year, fuel_type, category_id, branch_id)
VALUES (15, 'TN09YZ2015', 'Kia', 'Seltos', 2024, 'PETROL', 3, 2);
COMMIT;
SELECT vehicle_id, reg_number, brand, model, transmission, odometer, status, branch_id
FROM   vehicle WHERE vehicle_id = 15;
```

SCREENSHOT PLACEHOLDER – Figure 11

Delete this box and paste the screenshot (run block DML_INSERT of 06_dml_demo.sql). Listed as Figure 11 in the screenshot checklist.

Figure 11: *INSERT: new vehicle 15 added; omitted columns take their DEFAULT values*

```
-- 10% increment for the agents of branches 1 and 3 (the busiest branches in Q15)
SELECT emp_id, name, branch_id, salary FROM employee WHERE role = 'AGENT' AND branch_id IN (1, 3);
UPDATE employee SET salary = salary * 1.10 WHERE role = 'AGENT' AND branch_id IN (1, 3);
SELECT emp_id, name, branch_id, salary FROM employee WHERE role = 'AGENT' AND branch_id IN (1, 3);
-- Customer 7 changed phone number and address
UPDATE customer SET phone = '9841999107', address = 'Chromepet, Chennai' WHERE customer_id = 7;
SELECT customer_id, name, phone, address FROM customer WHERE customer_id = 7;
COMMIT;
```

SCREENSHOT PLACEHOLDER – Figure 12

Delete this box and paste the screenshot (run block DML_UPDATE of 06_dml_demo.sql). Listed as Figure 12 in the screenshot checklist.

Figure 12: *UPDATE: salary increment for two agents and change of a customer's contact details*

```
-- Booking 22 was cancelled. Deleting it also deletes its payments (ON DELETE CASCADE).
SELECT payment_id, booking_id, payment_type, amount FROM payment WHERE booking_id = 22;
DELETE FROM booking WHERE booking_id = 22 AND status = 'CANCELLED';
SELECT COUNT(*) AS payments_left FROM payment WHERE booking_id = 22;
COMMIT;
```

SCREENSHOT PLACEHOLDER – Figure 13

Delete this box and paste the screenshot (run block DML_DELETE of 06_dml_demo.sql). Listed as Figure 13 in the screenshot checklist.

Figure 13: *DELETE: cancelled booking 22 removed; its two payments are deleted by ON DELETE CASCADE*

Operation	Table	Effect
INSERT	vehicle	Vehicle 15 (Kia Seltos) added at branch 2 with default transmission, odometer and status.
UPDATE	employee	Salaries of agents 2 and 5 raised by 10%.
UPDATE	customer	Customer 7's phone and address changed (UNIQUE on phone still holds).
DELETE	booking → payment	Cancelled booking 22 removed; its 2 payments are removed automatically by ON DELETE CASCADE.

3 SQL Queries

The queries answer questions that branch agents and the head office actually ask (FR-03, FR-11, FR-12 of Review I). Each screenshot shows the query typed in SQL*Plus together with its output. Dates are displayed in the DD-MON-YY HH24:MI format set by 00_setup_display.sql.

Q1. Which vehicles can be rented in Chennai from 21-Oct 10:00 to 23-Oct 10:00? (FR-03)

Concepts: 3-table JOIN, correlated NOT EXISTS subquery, date range overlap | **Tables:** vehicle, vehicle_category, branch, booking

```
SELECT v.vehicle_id, v.reg_number, v.brand, v.model, vc.category_name, vc.daily_rate, b.branch_name
FROM   vehicle v
JOIN   vehicle_category vc ON vc.category_id = v.category_id
JOIN   branch b            ON b.branch_id    = v.branch_id
WHERE  b.city   = 'Chennai'
AND    v.status = 'AVAILABLE'
AND    NOT EXISTS (SELECT 1 FROM booking bk
                   WHERE  bk.vehicle_id = v.vehicle_id
                   AND    bk.status IN ('BOOKED', 'ONGOING')
                   AND    bk.start_date < TO_DATE('2026-10-23 10:00', 'YYYY-MM-DD HH24:MI')
                   AND    bk.end_date   > TO_DATE('2026-10-21 10:00', 'YYYY-MM-DD HH24:MI'))
ORDER  BY vc.daily_rate;
```

SCREENSHOT PLACEHOLDER – Figure 14

Delete this box and paste the screenshot (run block Q1 of 05_queries.sql). Listed as Figure 14 in the screenshot checklist.

Figure 14: *Q1 output: vehicles free in Chennai from 21-Oct to 23-Oct*

Vehicle 1 (Swift) is missing from the result because booking 19 overlaps these dates, and the retired Indica is filtered out by status.

Q2. Fleet status of every branch (FR-11, fleet utilisation)

Concepts: LEFT JOIN, COUNT, conditional aggregation with SUM(CASE) | **Tables:** branch, vehicle

```
SELECT b.branch_name,
       COUNT(v.vehicle_id)                                      AS fleet,
       SUM(CASE WHEN v.status = 'AVAILABLE'   THEN 1 ELSE 0 END) AS available,
       SUM(CASE WHEN v.status = 'RENTED'      THEN 1 ELSE 0 END) AS rented,
       SUM(CASE WHEN v.status = 'MAINTENANCE' THEN 1 ELSE 0 END) AS in_maint,
       SUM(CASE WHEN v.status = 'RETIRED'     THEN 1 ELSE 0 END) AS retired
FROM   branch b
LEFT JOIN vehicle v ON v.branch_id = b.branch_id
GROUP  BY b.branch_id, b.branch_name
ORDER  BY b.branch_id;
```

SCREENSHOT PLACEHOLDER – Figure 15

Delete this box and paste the screenshot (run block Q2 of 05_queries.sql). Listed as Figure 15 in the screenshot checklist.

Figure 15: *Q2 output: fleet status of every branch*

The LEFT JOIN keeps the Hyderabad Hitech City branch, which has no vehicle at the moment because its Fortuner was returned at Secunderabad.

Q3. Net revenue collected by each pickup branch (FR-11)

Concepts: 3-table JOIN, SUM with CASE (refunds subtracted), COUNT DISTINCT | **Tables:** payment, booking, branch

```
SELECT br.branch_name,
       COUNT(DISTINCT bk.booking_id) AS bookings,
       SUM(CASE WHEN p.payment_type = 'REFUND' THEN -p.amount ELSE p.amount END) AS net_revenue
FROM   payment p
JOIN   booking bk ON bk.booking_id = p.booking_id
JOIN   branch  br ON br.branch_id  = bk.pickup_branch_id
WHERE  p.txn_status = 'SUCCESS'
GROUP  BY br.branch_name
ORDER  BY net_revenue DESC;
```

SCREENSHOT PLACEHOLDER – Figure 16

Delete this box and paste the screenshot (run block Q3 of 05_queries.sql). Listed as Figure 16 in the screenshot checklist.

Figure 16: *Q3 output: net revenue collected by each pickup branch*

Q4. Month-wise collection by payment type (FR-11)

Concepts: TO_CHAR date function, GROUP BY expression, conditional SUM | **Tables:** payment

```
SELECT TO_CHAR(payment_date, 'YYYY-MM') AS month,
       SUM(CASE WHEN payment_type = 'ADVANCE'  THEN amount ELSE 0 END) AS advance,
       SUM(CASE WHEN payment_type = 'FINAL'    THEN amount ELSE 0 END) AS final,
       SUM(CASE WHEN payment_type = 'LATE_FEE' THEN amount ELSE 0 END) AS late_fee,
       SUM(CASE WHEN payment_type = 'REFUND'   THEN amount ELSE 0 END) AS refund
FROM   payment
WHERE  txn_status = 'SUCCESS'
GROUP  BY TO_CHAR(payment_date, 'YYYY-MM')
ORDER  BY month;
```

SCREENSHOT PLACEHOLDER – Figure 17

Delete this box and paste the screenshot (run block Q4 of 05_queries.sql). Listed as Figure 17 in the screenshot checklist.

Figure 17: *Q4 output: month-wise collection split by payment type*

Q5. Top three customers by amount paid (FR-11)

Concepts: JOIN, GROUP BY, ORDER BY inside an inline view, ROWNUM for top-N | **Tables:** customer, booking, payment

```
SELECT *
FROM  (SELECT c.customer_id, c.name, COUNT(DISTINCT bk.booking_id) AS bookings,
              SUM(p.amount) AS total_paid
       FROM   customer c
       JOIN   booking bk ON bk.customer_id = c.customer_id
       JOIN   payment p  ON p.booking_id   = bk.booking_id
       WHERE  p.txn_status = 'SUCCESS' AND p.payment_type <> 'REFUND'
       GROUP  BY c.customer_id, c.name
       ORDER  BY total_paid DESC)
WHERE  ROWNUM <= 3;
```

SCREENSHOT PLACEHOLDER – Figure 18

Delete this box and paste the screenshot (run block Q5 of 05_queries.sql). Listed as Figure 18 in the screenshot checklist.

Figure 18: *Q5 output: top three customers by amount paid*

Q6. Customers who paid more than the average customer

Concepts: Nested subquery in HAVING (average of per-customer totals) | **Tables:** customer, booking, payment

```
SELECT c.customer_id, c.name, SUM(p.amount) AS total_paid
FROM   customer c
JOIN   booking bk ON bk.customer_id = c.customer_id
JOIN   payment p  ON p.booking_id   = bk.booking_id
WHERE  p.txn_status = 'SUCCESS' AND p.payment_type <> 'REFUND'
GROUP  BY c.customer_id, c.name
HAVING SUM(p.amount) > (SELECT AVG(t.spent)
                        FROM  (SELECT SUM(p2.amount) AS spent
                               FROM   payment p2
                               JOIN   booking b2 ON b2.booking_id = p2.booking_id
                               WHERE  p2.txn_status = 'SUCCESS' AND p2.payment_type <> 'REFUND'
                               GROUP  BY b2.customer_id) t)
ORDER  BY total_paid DESC;
```

SCREENSHOT PLACEHOLDER – Figure 19

Delete this box and paste the screenshot (run block Q6 of 05_queries.sql). Listed as Figure 19 in the screenshot checklist.

Figure 19: *Q6 output: customers who paid more than the average customer*

Q7. Registered customers who never made a booking

Concepts: Set operation MINUS | **Tables:** customer, booking

```
SELECT customer_id, name, phone FROM customer
MINUS
SELECT c.customer_id, c.name, c.phone
FROM   customer c JOIN booking bk ON bk.customer_id = c.customer_id;
```

SCREENSHOT PLACEHOLDER – Figure 20

Delete this box and paste the screenshot (run block Q7 of 05_queries.sql). Listed as Figure 20 in the screenshot checklist.

Figure 20: *Q7 output: registered customers who have never made a booking*

Q8. Rentals, average rental length and average distance per category

Concepts: Two LEFT JOINs, COUNT DISTINCT, AVG of date difference and odometer difference | **Tables:** vehicle_category, vehicle, booking

```
SELECT vc.category_name,
       COUNT(DISTINCT v.vehicle_id)                    AS vehicles,
       COUNT(bk.booking_id)                            AS rentals,
       ROUND(AVG(bk.end_date - bk.start_date), 1)      AS avg_days,
       ROUND(AVG(bk.end_odometer - bk.start_odometer)) AS avg_km
FROM   vehicle_category vc
LEFT JOIN vehicle v  ON v.category_id = vc.category_id
LEFT JOIN booking bk ON bk.vehicle_id = v.vehicle_id AND bk.status = 'COMPLETED'
GROUP  BY vc.category_name
ORDER  BY rentals DESC, vc.category_name;
```

SCREENSHOT PLACEHOLDER – Figure 21

Delete this box and paste the screenshot (run block Q8 of 05_queries.sql). Listed as Figure 21 in the screenshot checklist.

Figure 21: *Q8 output: rentals, average days and average distance per category*

Q9. One-way rentals with pickup and return branch names (BR-07)

Concepts: Self-join: BRANCH joined twice with different aliases | **Tables:** booking, customer, vehicle, branch

```
SELECT bk.booking_id, c.name AS customer, v.reg_number,
       pb.branch_name AS pickup_branch, rb.branch_name AS return_branch, bk.status
FROM   booking bk
JOIN   customer c ON c.customer_id = bk.customer_id
JOIN   vehicle  v ON v.vehicle_id  = bk.vehicle_id
JOIN   branch  pb ON pb.branch_id  = bk.pickup_branch_id
JOIN   branch  rb ON rb.branch_id  = bk.return_branch_id
WHERE  bk.pickup_branch_id <> bk.return_branch_id
ORDER  BY bk.booking_id;
```

SCREENSHOT PLACEHOLDER – Figure 22

Delete this box and paste the screenshot (run block Q9 of 05_queries.sql). Listed as Figure 22 in the screenshot checklist.

Figure 22: *Q9 output: one-way rentals with pickup and return branch names*

Q10. Late returns, hours late and late fee billed (BR-05)

Concepts: Date arithmetic (minutes → started hours), LEFT JOIN with extra join condition, GROUP BY | **Tables:** booking, customer, vehicle, vehicle_category, payment

```
SELECT bk.booking_id, c.name AS customer, bk.end_date, bk.actual_return_date,
       CEIL(ROUND((bk.actual_return_date - bk.end_date) * 1440) / 60) AS hours_late,
       vc.late_fee_per_hr,
       NVL(SUM(p.amount), 0) AS fee_billed
FROM   booking bk
JOIN   customer c          ON c.customer_id  = bk.customer_id
JOIN   vehicle v           ON v.vehicle_id   = bk.vehicle_id
JOIN   vehicle_category vc ON vc.category_id = v.category_id
LEFT JOIN payment p        ON p.booking_id   = bk.booking_id AND p.payment_type = 'LATE_FEE'
WHERE  bk.actual_return_date > bk.end_date
GROUP  BY bk.booking_id, c.name, bk.end_date, bk.actual_return_date, vc.late_fee_per_hr
ORDER  BY bk.booking_id;
```

SCREENSHOT PLACEHOLDER – Figure 23

Delete this box and paste the screenshot (run block Q10 of 05_queries.sql). Listed as Figure 23 in the screenshot checklist.

Figure 23: *Q10 output: late returns, hours late and late fee billed*

HOURS_LATE × LATE_FEE_PER_HR equals FEE_BILLED for every row, which shows that the stored late-fee payments follow BR-05.

Q11. Payments that are pending or failed (pending dues, FR-11)

Concepts: 4-table JOIN, IN list | **Tables:** payment, booking, customer, vehicle

```
SELECT p.payment_id, p.booking_id, c.name AS customer, v.reg_number,
       p.payment_type, p.amount, p.txn_status
FROM   payment p
JOIN   booking  bk ON bk.booking_id = p.booking_id
JOIN   customer c  ON c.customer_id = bk.customer_id
JOIN   vehicle  v  ON v.vehicle_id  = bk.vehicle_id
WHERE  p.txn_status IN ('PENDING', 'FAILED')
ORDER  BY p.payment_id;
```

SCREENSHOT PLACEHOLDER – Figure 24

Delete this box and paste the screenshot (run block Q11 of 05_queries.sql). Listed as Figure 24 in the screenshot checklist.

Figure 24: *Q11 output: payments that are pending or failed*

Q12. Cancelled bookings with advance and refund (BR-06)

Concepts: LEFT JOIN, MAX and SUM with CASE | **Tables:** booking, customer, payment

```
SELECT bk.booking_id, c.name AS customer, bk.start_date,
       MAX(CASE WHEN p.payment_type = 'REFUND' THEN p.payment_date END) AS cancelled_on,
       SUM(CASE WHEN p.payment_type = 'ADVANCE' THEN p.amount ELSE 0 END) AS advance,
       SUM(CASE WHEN p.payment_type = 'REFUND'  THEN p.amount ELSE 0 END) AS refund
FROM   booking bk
JOIN   customer c ON c.customer_id = bk.customer_id
LEFT JOIN payment p ON p.booking_id = bk.booking_id
WHERE  bk.status = 'CANCELLED'
GROUP  BY bk.booking_id, c.name, bk.start_date
ORDER  BY bk.booking_id;
```

SCREENSHOT PLACEHOLDER – Figure 25

Delete this box and paste the screenshot (run block Q12 of 05_queries.sql). Listed as Figure 25 in the screenshot checklist.

Figure 25: *Q12 output: cancelled bookings with advance and refund amounts*

Booking 21 was cancelled more than 24 hours before pickup and got a full refund; booking 22 was cancelled five hours before and got 50%.

Q13. Vehicles whose maintenance cost so far is above Rs 5000 (FR-09)

Concepts: LEFT JOIN, NVL, string concatenation, HAVING | **Tables:** vehicle, maintenance

```
SELECT v.reg_number, v.brand || ' ' || v.model AS vehicle,
       COUNT(m.maint_id)  AS jobs,
       NVL(SUM(m.cost), 0) AS total_cost
FROM   vehicle v
LEFT JOIN maintenance m ON m.vehicle_id = v.vehicle_id
GROUP  BY v.reg_number, v.brand, v.model
HAVING NVL(SUM(m.cost), 0) > 5000
ORDER  BY total_cost DESC;
```

SCREENSHOT PLACEHOLDER – Figure 26

Delete this box and paste the screenshot (run block Q13 of 05_queries.sql). Listed as Figure 26 in the screenshot checklist.

Figure 26: *Q13 output: vehicles whose total maintenance cost is above Rs 5000*

Q14. Vehicles that are out of service right now, with the reason

Concepts: UNION ALL of two joins, string concatenation, TO_CHAR | **Tables:** vehicle, booking, customer, maintenance

```
SELECT v.reg_number, v.brand || ' ' || v.model AS vehicle, 'RENTED' AS state,
       'With ' || c.name || ', due ' || TO_CHAR(bk.end_date, 'DD-Mon HH24:MI') AS details
FROM   vehicle v
JOIN   booking  bk ON bk.vehicle_id = v.vehicle_id AND bk.status = 'ONGOING'
JOIN   customer c  ON c.customer_id = bk.customer_id
UNION ALL
SELECT v.reg_number, v.brand || ' ' || v.model, 'MAINTENANCE',
       m.service_type || ': ' || m.description
FROM   vehicle v
JOIN   maintenance m ON m.vehicle_id = v.vehicle_id AND m.maint_status = 'IN_PROGRESS';
```

SCREENSHOT PLACEHOLDER – Figure 27

Delete this box and paste the screenshot (run block Q14 of 05_queries.sql). Listed as Figure 27 in the screenshot checklist.

Figure 27: *Q14 output: vehicles that are currently out of service and why*

Q15. Agents who handled two or more bookings

Concepts: JOIN and LEFT JOIN, WHERE before grouping, HAVING after grouping | **Tables:** employee, branch, booking

```
SELECT e.emp_id, e.name AS employee, br.branch_name, COUNT(bk.booking_id) AS bookings_handled
FROM   employee e
JOIN   branch br ON br.branch_id = e.branch_id
LEFT JOIN booking bk ON bk.emp_id = e.emp_id
WHERE  e.role = 'AGENT'
GROUP  BY e.emp_id, e.name, br.branch_name
HAVING COUNT(bk.booking_id) >= 2
ORDER  BY bookings_handled DESC, e.emp_id;
```

SCREENSHOT PLACEHOLDER – Figure 28

Delete this box and paste the screenshot (run block Q15 of 05_queries.sql). Listed as Figure 28 in the screenshot checklist.

Figure 28: *Q15 output: agents who handled two or more bookings*

Q16. Customers whose licence expires within the next 12 months (BR-01 follow-up)

Concepts: SYSDATE, ADD_MONTHS, BETWEEN, TRUNC of a date difference | **Tables:** customer

```
SELECT customer_id, name, phone, license_no, license_expiry,
       TRUNC(license_expiry - SYSDATE) AS days_left
FROM   customer
WHERE  license_expiry BETWEEN SYSDATE AND ADD_MONTHS(SYSDATE, 12)
ORDER  BY license_expiry;
```

SCREENSHOT PLACEHOLDER – Figure 29

Delete this box and paste the screenshot (run block Q16 of 05_queries.sql). Listed as Figure 29 in the screenshot checklist.

Figure 29: *Q16 output: customers whose licence expires in the next 12 months*

These customers can still rent, but trg_booking_validate will refuse any booking that ends after their expiry date (Section 5).

4 Procedures and Functions

Functions return one value and are used inside SQL statements, procedures and triggers. Procedures carry out a complete business operation: they check the rules, change the data and end with COMMIT. A rule violation is reported with RAISE_APPLICATION_ERROR, so nothing is saved and the user sees a clear message. All objects are created with CREATE OR REPLACE and compiled without errors.

SCREENSHOT PLACEHOLDER – Figure 30

Delete this box and paste the screenshot (run block T_OBJECTS of 11_transactions.sql). Listed as Figure 30 in the screenshot checklist.

Figure 30: *All functions, procedures and triggers compiled with status VALID*

4.1 Functions (07_functions.sql)

Function	Returns	Used by
fn_age	Age in completed years on a date (derived attribute of CUSTOMER)	both validation triggers
fn_rental_days	Rental days, part of a day = full day, minimum 1 (derived attribute of BOOKING)	fn_calculate_rent
fn_calculate_rent	Rental days × daily rate (BR-04)	sp_create_booking, sp_return_vehicle
fn_late_fee	Started hours after end_date × late_fee_per_hr (BR-05)	sp_return_vehicle, fn_booking_balance
fn_is_vehicle_free	1 if no BOOKED/ONGOING booking overlaps the period, else 0 (BR-02)	sp_create_booking
fn_booking_balance	Rent + late fee − successful payments of a completed booking	sp_pending_dues (cursor)

```
-- Derived attribute "age" of CUSTOMER (Review I): age in completed years on a given date.
CREATE OR REPLACE FUNCTION fn_age (p_dob IN DATE, p_on IN DATE)
RETURN NUMBER
IS
BEGIN
  RETURN TRUNC(MONTHS_BETWEEN(p_on, p_dob) / 12);
END fn_age;
/
```

```
-- Derived attribute "rental_days" of BOOKING (BR-04): part of a day counts as a full day,
-- minimum one day. Minutes are rounded first so DATE arithmetic cannot add a false extra day.
CREATE OR REPLACE FUNCTION fn_rental_days (p_start IN DATE, p_end IN DATE)
RETURN NUMBER
IS
BEGIN
  RETURN GREATEST(1, CEIL(ROUND((p_end - p_start) * 1440) / 1440));
END fn_rental_days;
/
```

```
-- Rent of a booking = rental days x daily rate of the vehicle's category (BR-04).
CREATE OR REPLACE FUNCTION fn_calculate_rent (p_booking_id IN booking.booking_id%TYPE)
RETURN NUMBER
IS
  v_rent NUMBER;
BEGIN
  SELECT fn_rental_days(b.start_date, b.end_date) * vc.daily_rate
  INTO   v_rent
  FROM   booking b
  JOIN   vehicle v           ON v.vehicle_id   = b.vehicle_id
  JOIN   vehicle_category vc ON vc.category_id = v.category_id
  WHERE  b.booking_id = p_booking_id;
  RETURN v_rent;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RETURN NULL;
END fn_calculate_rent;
/
```

```
-- Late fee (BR-05): every started hour after end_date x late_fee_per_hr. 0 if on time or not returned.
```

```
CREATE OR REPLACE FUNCTION fn_late_fee (p_booking_id IN booking.booking_id%TYPE)
RETURN NUMBER
IS
  v_minutes NUMBER;
  v_rate    vehicle_category.late_fee_per_hr%TYPE;
BEGIN
  SELECT ROUND((b.actual_return_date - b.end_date) * 1440), vc.late_fee_per_hr
  INTO   v_minutes, v_rate
  FROM   booking b
  JOIN   vehicle v           ON v.vehicle_id   = b.vehicle_id
  JOIN   vehicle_category vc ON vc.category_id = v.category_id
  WHERE  b.booking_id = p_booking_id;
  IF v_minutes IS NULL OR v_minutes <= 0 THEN
    RETURN 0;
  END IF;
  RETURN CEIL(v_minutes / 60) * v_rate;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RETURN NULL;
END fn_late_fee;
/
```

```
-- BR-02: returns 1 if the vehicle has no BOOKED/ONGOING booking overlapping the period, else 0.
CREATE OR REPLACE FUNCTION fn_is_vehicle_free (p_vehicle_id IN vehicle.vehicle_id%TYPE,
                                               p_start      IN DATE,
                                               p_end        IN DATE)
RETURN NUMBER
IS
  v_count NUMBER;
BEGIN
  SELECT COUNT(*)
  INTO   v_count
  FROM   booking
  WHERE  vehicle_id = p_vehicle_id
  AND    status IN ('BOOKED', 'ONGOING')
  AND    start_date < p_end
  AND    end_date   > p_start;
  IF v_count = 0 THEN
    RETURN 1;
  END IF;
  RETURN 0;
END fn_is_vehicle_free;
/
```

```
-- Amount still to be collected for a COMPLETED booking:
-- rent + late fee - successful advance/final/late-fee payments. 0 for other bookings.
CREATE OR REPLACE FUNCTION fn_booking_balance (p_booking_id IN booking.booking_id%TYPE)
RETURN NUMBER
IS
  v_status booking.status%TYPE;
  v_paid   NUMBER;
BEGIN
  SELECT status INTO v_status FROM booking WHERE booking_id = p_booking_id;
  IF v_status <> 'COMPLETED' THEN
    RETURN 0;
  END IF;
  SELECT NVL(SUM(amount), 0)
  INTO   v_paid
  FROM   payment
  WHERE  booking_id = p_booking_id
  AND    txn_status = 'SUCCESS'
  AND    payment_type IN ('ADVANCE', 'FINAL', 'LATE_FEE');
  RETURN fn_calculate_rent(p_booking_id) + fn_late_fee(p_booking_id) - v_paid;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RETURN NULL;
END fn_booking_balance;
/
```

SCREENSHOT PLACEHOLDER – Figure 31

Delete this box and paste the screenshot (run block T_FUNCS1 of 11_transactions.sql). Listed as Figure 31 in the screenshot checklist.

Figure 31: *fn_rental_days, fn_calculate_rent, fn_late_fee and fn_booking_balance used in a query*

For booking 2 the rent is 3 days × 2800 = 8400 and the late fee is 4 hours × 250 = 1000. Bookings 12 and 16 still have a balance because one payment is PENDING and one FAILED.

SCREENSHOT PLACEHOLDER – Figure 32

Delete this box and paste the screenshot (run block T_FUNCS2 of 11_transactions.sql). Listed as Figure 32 in the screenshot checklist.

Figure 32: *fn_age for three customers and fn_is_vehicle_free for two vehicles*

4.2 Procedures (08_procedures.sql)

Procedure	Requirement	Tables changed	What it does
sp_register_customer	FR-01	customer	Adds a customer with the next sequence ID; duplicates give ORA-20150.
sp_create_booking	FR-04, FR-07	booking, payment	Checks branch and overlap, inserts the booking and the advance.
sp_pickup_vehicle	FR-05	booking (vehicle by trigger)	BOOKED → ONGOING and copies the odometer.
sp_return_vehicle	FR-06, FR-07	booking, payment (vehicle by trigger)	Records return, computes rent and late fee, inserts FINAL and LATE_FEE.
sp_cancel_booking	FR-08	booking, payment	BOOKED → CANCELLED and inserts the refund by BR-06.

sp_register_customer

```
-- FR-01: register a customer. Age and licence checks are done by trigger trg_customer_validate.
CREATE OR REPLACE PROCEDURE sp_register_customer (
  p_name           IN customer.name%TYPE,
  p_phone          IN customer.phone%TYPE,
  p_email          IN customer.email%TYPE,
  p_license_no     IN customer.license_no%TYPE,
  p_license_expiry IN customer.license_expiry%TYPE,
  p_dob            IN customer.dob%TYPE,
  p_address        IN customer.address%TYPE DEFAULT NULL
) IS
  v_id customer.customer_id%TYPE;
BEGIN
  v_id := seq_customer.NEXTVAL;
  INSERT INTO customer (customer_id, name, phone, email, license_no, license_expiry, dob, address,
registered_on)
  VALUES (v_id, p_name, p_phone, p_email, p_license_no, p_license_expiry, p_dob, p_address, SYSDATE);
  COMMIT;
  DBMS_OUTPUT.PUT_LINE('Customer registered. ID = ' || v_id);
EXCEPTION
  WHEN DUP_VAL_ON_INDEX THEN
    RAISE_APPLICATION_ERROR(-20150, 'Phone, email or licence number is already registered');
END sp_register_customer;
/
```

SCREENSHOT PLACEHOLDER – Figure 33

Delete this box and paste the screenshot (run block T_REGISTER of 11_transactions.sql). Listed as Figure 33 in the screenshot checklist.

Figure 33: *sp_register_customer: customer 13 registered*

sp_create_booking

```
-- FR-04 / FR-07: create a booking and record the advance payment.
```

```
-- BR-02 (no overlap) is checked here with fn_is_vehicle_free; BR-01 and BR-03 by trg_booking_validate.
CREATE OR REPLACE PROCEDURE sp_create_booking (
  p_customer_id   IN booking.customer_id%TYPE,
  p_vehicle_id    IN booking.vehicle_id%TYPE,
  p_emp_id        IN booking.emp_id%TYPE,
  p_pickup_branch IN booking.pickup_branch_id%TYPE,
  p_return_branch IN booking.return_branch_id%TYPE,
  p_start_date    IN DATE,
  p_end_date      IN DATE,
  p_advance       IN payment.amount%TYPE DEFAULT 0,
  p_method        IN payment.method%TYPE DEFAULT 'CASH'
) IS
  v_branch  vehicle.branch_id%TYPE;
  v_booking booking.booking_id%TYPE;
BEGIN
  SELECT branch_id INTO v_branch FROM vehicle WHERE vehicle_id = p_vehicle_id;
  IF v_branch <> p_pickup_branch THEN
    RAISE_APPLICATION_ERROR(-20102, 'Vehicle ' || p_vehicle_id || ' is at branch ' || v_branch ||
                                    ', not at pickup branch ' || p_pickup_branch);
  END IF;
  IF fn_is_vehicle_free(p_vehicle_id, p_start_date, p_end_date) = 0 THEN
    RAISE_APPLICATION_ERROR(-20103, 'Vehicle ' || p_vehicle_id || ' is already booked for these dates');
  END IF;

  v_booking := seq_booking.NEXTVAL;
  INSERT INTO booking (booking_id, customer_id, vehicle_id, emp_id, pickup_branch_id, return_branch_id,
                       booking_date, start_date, end_date, status)
  VALUES (v_booking, p_customer_id, p_vehicle_id, p_emp_id, p_pickup_branch, p_return_branch,
          SYSDATE, p_start_date, p_end_date, 'BOOKED');

  IF p_advance > 0 THEN
    INSERT INTO payment (payment_id, booking_id, amount, payment_date, method, payment_type, txn_status)
    VALUES (seq_payment.NEXTVAL, v_booking, p_advance, SYSDATE, p_method, 'ADVANCE', 'SUCCESS');
  END IF;
  COMMIT;
  DBMS_OUTPUT.PUT_LINE('Booking ' || v_booking || ' created for vehicle ' || p_vehicle_id ||
                       '. Estimated rent: Rs ' || fn_calculate_rent(v_booking) ||
                       ', advance paid: Rs ' || p_advance);
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RAISE_APPLICATION_ERROR(-20101, 'Vehicle ' || p_vehicle_id || ' does not exist');
END sp_create_booking;
/
```

SCREENSHOT PLACEHOLDER – Figure 34

Delete this box and paste the screenshot (run block T_BOOK of 11_transactions.sql). Listed as Figure 34 in the screenshot checklist.

Figure 34: *sp_create_booking: booking 23 created with a Rs 400 advance*

Booking 23 is a one-way rental (pickup at branch 1, return at branch 2). The estimated rent is 2 days × Rs 500 for the Scooter category.

sp_pickup_vehicle

```
-- FR-05: customer collects the vehicle. start_odometer is copied from the vehicle.
-- trg_booking_status_sync then marks the vehicle RENTED.
CREATE OR REPLACE PROCEDURE sp_pickup_vehicle (p_booking_id IN booking.booking_id%TYPE)
IS
  v_status  booking.status%TYPE;
  v_vstatus vehicle.status%TYPE;
  v_odo     vehicle.odometer%TYPE;
BEGIN
  SELECT b.status, v.status, v.odometer
  INTO   v_status, v_vstatus, v_odo
  FROM   booking b JOIN vehicle v ON v.vehicle_id = b.vehicle_id
  WHERE  b.booking_id = p_booking_id;
  IF v_status <> 'BOOKED' THEN
    RAISE_APPLICATION_ERROR(-20120, 'Booking ' || p_booking_id || ' is ' || v_status || ', not BOOKED');
  END IF;
  IF v_vstatus <> 'AVAILABLE' THEN
    RAISE_APPLICATION_ERROR(-20122, 'Vehicle is ' || v_vstatus || ' and cannot be handed over');
  END IF;
  UPDATE booking SET status = 'ONGOING', start_odometer = v_odo WHERE booking_id = p_booking_id;
  COMMIT;
```

```
  DBMS_OUTPUT.PUT_LINE('Booking ' || p_booking_id || ' is now ONGOING. Start odometer: ' || v_odo);
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RAISE_APPLICATION_ERROR(-20121, 'Booking ' || p_booking_id || ' not found');
END sp_pickup_vehicle;
/
```

SCREENSHOT PLACEHOLDER – Figure 35

Delete this box and paste the screenshot (run block T_PICKUP of 11_transactions.sql). Listed as Figure 35 in the screenshot checklist.

Figure 35: *sp_pickup_vehicle: booking 23 becomes ONGOING and the vehicle becomes RENTED*

The procedure only updates BOOKING. The vehicle status changes to RENTED because of trigger trg_booking_status_sync (Section 5.2).

sp_return_vehicle

```
-- FR-06 / FR-07: vehicle returned. Computes rent and late fee, records FINAL and LATE_FEE payments.
-- trg_booking_status_sync makes the vehicle AVAILABLE at the return branch (BR-07).
CREATE OR REPLACE PROCEDURE sp_return_vehicle (
  p_booking_id   IN booking.booking_id%TYPE,
  p_end_odometer IN booking.end_odometer%TYPE,
  p_return_time  IN DATE DEFAULT SYSDATE,
  p_method       IN payment.method%TYPE DEFAULT 'CASH'
) IS
  v_status    booking.status%TYPE;
  v_start_odo booking.start_odometer%TYPE;
  v_rent      NUMBER;
  v_late      NUMBER;
  v_advance   NUMBER;
  v_due       NUMBER;
BEGIN
  SELECT status, start_odometer INTO v_status, v_start_odo
  FROM   booking WHERE booking_id = p_booking_id;
  IF v_status <> 'ONGOING' THEN
    RAISE_APPLICATION_ERROR(-20130, 'Booking ' || p_booking_id || ' is ' || v_status || ', not ONGOING');
  END IF;
  IF p_end_odometer < v_start_odo THEN
    RAISE_APPLICATION_ERROR(-20131, 'End odometer cannot be less than start odometer ' || v_start_odo);
  END IF;

  UPDATE booking
  SET    actual_return_date = p_return_time, end_odometer = p_end_odometer, status = 'COMPLETED'
  WHERE  booking_id = p_booking_id;

  v_rent := fn_calculate_rent(p_booking_id);
  v_late := fn_late_fee(p_booking_id);
  SELECT NVL(SUM(amount), 0) INTO v_advance
  FROM   payment
  WHERE  booking_id = p_booking_id AND payment_type = 'ADVANCE' AND txn_status = 'SUCCESS';
  v_due := v_rent - v_advance;

  IF v_due > 0 THEN
    INSERT INTO payment (payment_id, booking_id, amount, payment_date, method, payment_type, txn_status)
    VALUES (seq_payment.NEXTVAL, p_booking_id, v_due, p_return_time, p_method, 'FINAL', 'SUCCESS');
  END IF;
  IF v_late > 0 THEN
    INSERT INTO payment (payment_id, booking_id, amount, payment_date, method, payment_type, txn_status)
    VALUES (seq_payment.NEXTVAL, p_booking_id, v_late, p_return_time, p_method, 'LATE_FEE', 'SUCCESS');
  END IF;
  COMMIT;
  DBMS_OUTPUT.PUT_LINE('Booking ' || p_booking_id || ' completed. Rent: Rs ' || v_rent ||
                       ', advance: Rs ' || v_advance || ', final payment: Rs ' || GREATEST(v_due, 0) ||
                       ', late fee: Rs ' || v_late);
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RAISE_APPLICATION_ERROR(-20132, 'Booking ' || p_booking_id || ' not found');
END sp_return_vehicle;
/
```

SCREENSHOT PLACEHOLDER – Figure 36

Delete this box and paste the screenshot (run block T_RETURN of 11_transactions.sql). Listed as Figure 36 in the screenshot checklist.

Figure 36: *sp_return_vehicle: rent, final payment and late fee for a late one-way return*

The return time is passed as a parameter so that a late return can be shown on the same day. It is 3 hours late, so the late fee is 3 × Rs 50 = Rs 150. The trigger moved vehicle 12 to branch 2 (BR-07) and updated its odometer to 9120.

sp_cancel_booking

```
-- FR-08 / BR-06: cancel a BOOKED booking. Full refund of the advance if cancelled 24 hours or more
-- before pickup, otherwise 50%. The refund goes back by the same method as the advance.
CREATE OR REPLACE PROCEDURE sp_cancel_booking (
  p_booking_id  IN booking.booking_id%TYPE,
  p_cancel_time IN DATE DEFAULT SYSDATE
) IS
  v_status  booking.status%TYPE;
  v_start   booking.start_date%TYPE;
  v_advance NUMBER;
  v_method  payment.method%TYPE;
  v_refund  NUMBER;
BEGIN
  SELECT status, start_date INTO v_status, v_start FROM booking WHERE booking_id = p_booking_id;
  IF v_status <> 'BOOKED' THEN
    RAISE_APPLICATION_ERROR(-20140, 'Only a BOOKED booking can be cancelled. Current status: ' || v_status);
  END IF;

  SELECT NVL(SUM(amount), 0), MIN(method) INTO v_advance, v_method
  FROM   payment
  WHERE  booking_id = p_booking_id AND payment_type = 'ADVANCE' AND txn_status = 'SUCCESS';

  IF v_start - p_cancel_time >= 1 THEN
    v_refund := v_advance;                -- 24 hours or more before pickup
  ELSE
    v_refund := ROUND(v_advance * 0.5, 2); -- late cancellation
  END IF;

  UPDATE booking SET status = 'CANCELLED' WHERE booking_id = p_booking_id;
  IF v_refund > 0 THEN
    INSERT INTO payment (payment_id, booking_id, amount, payment_date, method, payment_type, txn_status)
    VALUES (seq_payment.NEXTVAL, p_booking_id, v_refund, p_cancel_time, v_method, 'REFUND', 'SUCCESS');
  END IF;
  COMMIT;
  DBMS_OUTPUT.PUT_LINE('Booking ' || p_booking_id || ' cancelled. Advance: Rs ' || v_advance ||
                       ', refund: Rs ' || v_refund);
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RAISE_APPLICATION_ERROR(-20141, 'Booking ' || p_booking_id || ' not found');
END sp_cancel_booking;
/
```

SCREENSHOT PLACEHOLDER – Figure 37

Delete this box and paste the screenshot (run block T_CANCEL of 11_transactions.sql). Listed as Figure 37 in the screenshot checklist.

Figure 37: sp_cancel_booking: full refund for early cancellation, 50% for late cancellation

5 Cursors and Triggers

5.1 Cursors (09_cursors.sql)

sp_overdue_report uses an explicit cursor. It OPENs the cursor, FETCHes one overdue rental at a time, works out how many hours late it is and the fee so far, and stops when %NOTFOUND becomes true. %ROWCOUNT gives the number of rows processed, and the cursor is then CLOSEd. This answers the Review I problem of vehicles "quietly going overdue".

```
-- Explicit cursor: lists every ONGOING booking whose end date has passed (vehicle not returned)
-- and the late fee building up so far (BR-05). Uses OPEN, FETCH, CLOSE, %NOTFOUND and %ROWCOUNT.
CREATE OR REPLACE PROCEDURE sp_overdue_report
IS
  CURSOR c_overdue IS
    SELECT b.booking_id, c.name, c.phone, v.reg_number, b.end_date, vc.late_fee_per_hr
    FROM   booking b
    JOIN   customer c          ON c.customer_id  = b.customer_id
    JOIN   vehicle v           ON v.vehicle_id   = b.vehicle_id
    JOIN   vehicle_category vc ON vc.category_id = v.category_id
    WHERE  b.status = 'ONGOING'
    AND    b.end_date < SYSDATE
    ORDER  BY b.end_date;
  r_row   c_overdue%ROWTYPE;
  v_hours NUMBER;
  v_fee   NUMBER;
  v_total NUMBER := 0;
BEGIN
  OPEN c_overdue;
  LOOP
    FETCH c_overdue INTO r_row;
    EXIT WHEN c_overdue%NOTFOUND;
    v_hours := CEIL(ROUND((SYSDATE - r_row.end_date) * 1440) / 60);
    v_fee   := v_hours * r_row.late_fee_per_hr;
    v_total := v_total + v_fee;
    DBMS_OUTPUT.PUT_LINE('Booking ' || r_row.booking_id || ' | ' || r_row.reg_number || ' | ' ||
                         r_row.name || ' (' || r_row.phone || ') | due ' ||
                         TO_CHAR(r_row.end_date, 'DD-Mon HH24:MI') || ' | ' ||
                         v_hours || ' h late | fee so far Rs ' || v_fee);
  END LOOP;
  DBMS_OUTPUT.PUT_LINE(c_overdue%ROWCOUNT || ' overdue booking(s). Total late fee so far: Rs ' || v_total);
  CLOSE c_overdue;
END sp_overdue_report;
/
```

SCREENSHOT PLACEHOLDER – Figure 38

Delete this box and paste the screenshot (run block T_OVERDUE of 11_transactions.sql). Listed as Figure 38 in the screenshot checklist.

Figure 38: Explicit cursor (sp_overdue_report): rentals past their end time

The output depends on the date and time the procedure is run, because the hours late are counted up to SYSDATE.

sp_pending_dues uses a cursor FOR loop, which opens, fetches and closes the cursor automatically. For every completed booking it calls fn_booking_balance and prints only those that still owe money.

```
-- Cursor FOR loop: goes through all COMPLETED bookings and prints the ones that still have
-- money to collect (FR-11 "pending dues"). The balance comes from fn_booking_balance.
CREATE OR REPLACE PROCEDURE sp_pending_dues
IS
  CURSOR c_completed IS
    SELECT b.booking_id, c.name, c.phone
    FROM   booking b JOIN customer c ON c.customer_id = b.customer_id
    WHERE  b.status = 'COMPLETED'
    ORDER  BY b.booking_id;
  v_balance NUMBER;
  v_count   NUMBER := 0;
  v_total   NUMBER := 0;
BEGIN
  FOR r IN c_completed LOOP
    v_balance := fn_booking_balance(r.booking_id);
    IF v_balance > 0 THEN
      v_count := v_count + 1;
      v_total := v_total + v_balance;
      DBMS_OUTPUT.PUT_LINE('Booking ' || r.booking_id || ' | ' || r.name || ' (' || r.phone ||
```

```
                          ') | balance Rs ' || v_balance);
    END IF;
  END LOOP;
  DBMS_OUTPUT.PUT_LINE(v_count || ' booking(s) with pending dues. Total: Rs ' || v_total);
END sp_pending_dues;
/
```

SCREENSHOT PLACEHOLDER – Figure 39

Delete this box and paste the screenshot (run block T_DUES of 11_transactions.sql). Listed as Figure 39 in the screenshot checklist.

Figure 39: *Cursor FOR loop (sp_pending_dues): completed bookings with money still to collect*

5.2 Triggers (10_triggers.sql)

Trigger	Timing / event	Table	Rule
trg_customer_validate	BEFORE INSERT OR UPDATE OF dob, license_expiry	customer	BR-01 at registration
trg_booking_validate	BEFORE INSERT	booking	BR-01 for the rental period, BR-03
trg_booking_status_sync	AFTER UPDATE OF status	booking	Vehicle status, BR-07
trg_maintenance_sync	AFTER INSERT OR UPDATE OF maint_status	maintenance	FR-09, BR-03

All four are row-level triggers. None of them reads or changes the table it is defined on; they only touch CUSTOMER and VEHICLE. This avoids the mutating-table error (ORA-04091).

```
-- BR-01 at registration: customer must be 21 or older and the licence must not be expired.
-- A CHECK constraint cannot use SYSDATE, so this is done in a trigger (as planned in Review I).
CREATE OR REPLACE TRIGGER trg_customer_validate
BEFORE INSERT OR UPDATE OF dob, license_expiry ON customer
FOR EACH ROW
BEGIN
  IF fn_age(:NEW.dob, SYSDATE) < 21 THEN
    RAISE_APPLICATION_ERROR(-20201, 'Customer must be at least 21 years old. Age: ' ||
                                    fn_age(:NEW.dob, SYSDATE));
  END IF;
  IF :NEW.license_expiry <= SYSDATE THEN
    RAISE_APPLICATION_ERROR(-20202, 'Driving licence has expired on ' ||
                                    TO_CHAR(:NEW.license_expiry, 'DD-MON-YYYY'));
  END IF;
END;
/
```

```
-- BR-01 and BR-03 for every new booking: customer is 21+ on the start date, licence is valid
-- until the end date, and the vehicle is AVAILABLE (not RENTED, MAINTENANCE or RETIRED).
CREATE OR REPLACE TRIGGER trg_booking_validate
BEFORE INSERT ON booking
FOR EACH ROW
DECLARE
  v_dob     customer.dob%TYPE;
  v_expiry  customer.license_expiry%TYPE;
  v_vstatus vehicle.status%TYPE;
BEGIN
  SELECT dob, license_expiry INTO v_dob, v_expiry
  FROM   customer WHERE customer_id = :NEW.customer_id;
  IF fn_age(v_dob, :NEW.start_date) < 21 THEN
    RAISE_APPLICATION_ERROR(-20211, 'Customer is under 21 on the rental start date');
  END IF;
  IF v_expiry < :NEW.end_date THEN
    RAISE_APPLICATION_ERROR(-20212, 'Driving licence expires on ' || TO_CHAR(v_expiry, 'DD-MON-YYYY') ||
                                    ', before the rental ends');
  END IF;

  SELECT status INTO v_vstatus FROM vehicle WHERE vehicle_id = :NEW.vehicle_id;
  IF v_vstatus <> 'AVAILABLE' THEN
    RAISE_APPLICATION_ERROR(-20213, 'Vehicle ' || :NEW.vehicle_id || ' cannot be booked. Status: ' ||
v_vstatus);
  END IF;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RAISE_APPLICATION_ERROR(-20214, 'Customer or vehicle does not exist');
```

```
END;
/
```

```
-- Keeps VEHICLE in step with BOOKING: pickup makes the vehicle RENTED; return makes it
-- AVAILABLE, moves it to the return branch (BR-07) and updates its odometer.
CREATE OR REPLACE TRIGGER trg_booking_status_sync
AFTER UPDATE OF status ON booking
FOR EACH ROW
BEGIN
  IF :OLD.status = 'BOOKED' AND :NEW.status = 'ONGOING' THEN
    UPDATE vehicle SET status = 'RENTED' WHERE vehicle_id = :NEW.vehicle_id;
  ELSIF :OLD.status = 'ONGOING' AND :NEW.status = 'COMPLETED' THEN
    UPDATE vehicle
    SET    status    = 'AVAILABLE',
           branch_id = :NEW.return_branch_id,
           odometer  = NVL(:NEW.end_odometer, odometer)
    WHERE  vehicle_id = :NEW.vehicle_id;
  END IF;
END;
/
```

```
-- FR-09: a job IN_PROGRESS takes the vehicle out of the fleet (status MAINTENANCE);
-- a DONE job releases it. A vehicle that is out on rent cannot go into the workshop.
CREATE OR REPLACE TRIGGER trg_maintenance_sync
AFTER INSERT OR UPDATE OF maint_status ON maintenance
FOR EACH ROW
DECLARE
  v_vstatus vehicle.status%TYPE;
BEGIN
  IF :NEW.maint_status = 'IN_PROGRESS' THEN
    SELECT status INTO v_vstatus FROM vehicle WHERE vehicle_id = :NEW.vehicle_id;
    IF v_vstatus = 'RENTED' THEN
      RAISE_APPLICATION_ERROR(-20221, 'Vehicle ' || :NEW.vehicle_id || ' is out on rent; maintenance cannot
start');
    ELSIF v_vstatus = 'AVAILABLE' THEN
      UPDATE vehicle SET status = 'MAINTENANCE' WHERE vehicle_id = :NEW.vehicle_id;
    END IF;
  ELSIF :NEW.maint_status = 'DONE' THEN
    UPDATE vehicle SET status = 'AVAILABLE'
    WHERE  vehicle_id = :NEW.vehicle_id AND status = 'MAINTENANCE';
  END IF;
END;
/
```

5.3 Trigger tests

Action	Trigger fired	Effect in the database	Figure
Register an under-age customer / expired licence	trg_customer_validate	INSERT rejected (ORA-20201, ORA-20202)	Figure 40
Book with a licence that ends too early	trg_booking_validate	INSERT rejected (ORA-20212)	Figure 41
Book a vehicle that is RENTED	trg_booking_validate	INSERT rejected (ORA-20213)	Figure 41
sp_pickup_vehicle(23)	trg_booking_status_sync	Vehicle 12: AVAILABLE → RENTED	Figure 35
sp_return_vehicle(23, …)	trg_booking_status_sync	Vehicle 12: RENTED → AVAILABLE, branch 1 → 2, odometer 9120	Figure 36
Maintenance job 11 set to DONE	trg_maintenance_sync	Vehicle 8: MAINTENANCE → AVAILABLE	Figure 42
New IN_PROGRESS job for vehicle 3	trg_maintenance_sync	Vehicle 3: AVAILABLE → MAINTENANCE; booking it is refused	Figure 42
IN_PROGRESS job for rented vehicle 2	trg_maintenance_sync	INSERT rejected (ORA-20221)	Figure 42

SCREENSHOT PLACEHOLDER – Figure 40

Delete this box and paste the screenshot (run block T_TRG_CUSTOMER of 11_transactions.sql). Listed as Figure 40 in the screenshot checklist.

Figure 40: *trg_customer_validate rejects an under-age customer and an expired licence*

SCREENSHOT PLACEHOLDER – Figure 41

Delete this box and paste the screenshot (run block T_TRG_BOOKING of 11_transactions.sql). Listed as Figure 41 in the screenshot checklist.

Figure 41: *trg_booking_validate rejects a short licence and a vehicle that is already rented*

SCREENSHOT PLACEHOLDER – Figure 42

Delete this box and paste the screenshot (run block T_TRG_MAINT of 11_transactions.sql). Listed as Figure 42 in the screenshot checklist.

Figure 42: *trg_maintenance_sync: vehicle 8 released, vehicle 3 sent to the workshop, two blocked actions*

6 Real-World Transaction Flow

Flow followed by every operation: **User operation → SQL / PL/SQL → database processing → update or retrieval → result**.

User operation	Component executed	Tables	How the database changes	Result	Fig.
Agent registers a customer	sp_register_customer, trg_customer_validate	customer	New row after age/licence check	Customer ID	33
Agent books a vehicle	sp_create_booking, fn_is_vehicle_free, fn_calculate_rent, trg_booking_validate	booking, payment	Booking BOOKED + ADVANCE payment	Booking ID, rent	34
Customer collects the vehicle	sp_pickup_vehicle, trg_booking_status_sync	booking, vehicle	Booking ONGOING, vehicle RENTED	Start odometer	35

User operation	Component executed	Tables	How the database changes	Result	Fig.
Customer returns the vehicle	sp_return_vehicle, fn_calculate_rent, fn_late_fee, trg_booking_status_sync	booking, payment, vehicle	COMPLETED, FINAL and LATE_FEE rows, vehicle moved	Bill	36
Customer cancels	sp_cancel_booking	booking, payment	CANCELLED + REFUND row	Refund amount	37
Mechanic starts / closes a job	DML on maintenance, trg_maintenance_sync	maintenance, vehicle	Vehicle MAINTENANCE / AVAILABLE	Vehicle status	42
Manager checks overdue rentals and dues	sp_overdue_report, sp_pending_dues	booking, customer, vehicle, payment	Read only	Report lines	38, 39
Manager views reports	Q1–Q16	all tables	Read only	Result rows	14–29

6.1 Where each business rule of Review I is enforced

Rule	Enforced by	Error when broken	Evidence
BR-01 age ≥ 21, licence valid	trg_customer_validate, trg_booking_validate	ORA-20201, -20202, -20211, -20212	Figure 40, 41
BR-02 no overlapping bookings	fn_is_vehicle_free inside sp_create_booking	ORA-20103	Figure 32, 43
BR-03 only AVAILABLE vehicles booked	trg_booking_validate	ORA-20213	Figure 41, 42
BR-04 rent = days × rate, min 1 day	fn_rental_days, fn_calculate_rent	–	Figure 31
BR-05 late fee per started hour	fn_late_fee, sp_return_vehicle	–	Figure 23, 36
BR-06 refund 100% or 50%	sp_cancel_booking	ORA-20140 if not BOOKED	Figure 25, 37
BR-07 one-way return moves vehicle	trg_booking_status_sync	–	Figure 36
BR-08 end odometer ≥ start	ck_booking_odometer, sp_return_vehicle	ORA-02290, ORA-20131	Figure 7

SCREENSHOT PLACEHOLDER – Figure 43

Delete this box and paste the screenshot (run block T_RULES of 11_transactions.sql). Listed as Figure 43 in the screenshot checklist.

Figure 43: *Business-rule violations rejected by the procedures*

6.2 Implementation decisions

- **Overlap check in a procedure, not a trigger.** Checking BR-02 needs a SELECT on BOOKING. Doing that in a row-level trigger on BOOKING can raise the mutating-table error, so it is done in sp_create_booking with fn_is_vehicle_free.

- **Part of an hour or day is charged in full.** Review I says "each hour" and "minimum one day"; we charge every started hour and every started day. Minutes are rounded first so date arithmetic cannot add a false extra unit.
- **Advance amount is entered by the agent.** Review I does not fix a percentage, so the advance is a parameter of sp_create_booking.
- **A vehicle must be at its pickup branch.** This follows from BR-07: after a one-way rental the vehicle belongs to the return branch.
- **Sample data loaded before the triggers.** The history (completed and cancelled rentals) is inserted directly, so the triggers only act on new transactions.

7 Viva Preparation: SQL / PL/SQL Understanding

Likely question	Short answer
Why are age and licence checked by a trigger and not a CHECK constraint?	They depend on today's date. Oracle does not allow SYSDATE in a CHECK constraint, so trg_customer_validate does it before every insert.
Procedure vs function in this project?	Functions return one value and are used in SQL (fn_calculate_rent in Q-style SELECTs). Procedures do a full operation and change data (sp_return_vehicle updates BOOKING and inserts PAYMENT rows).
Why use sequences?	They give unique IDs safely when many agents book at the same time. MAX(id)+1 could give the same ID to two sessions.
Which cursors did you use?	An explicit cursor (OPEN, FETCH, CLOSE, %NOTFOUND, %ROWCOUNT) in sp_overdue_report and a cursor FOR loop in sp_pending_dues.
BEFORE vs AFTER trigger?	BEFORE triggers check data before it is written (validation triggers). AFTER triggers react to a change that has been made, e.g. trg_booking_status_sync updates VEHICLE after the booking status changes.
What are :NEW and :OLD?	The new and old values of the row being changed. trg_booking_status_sync uses :OLD.status = 'ONGOING' and :NEW.status = 'COMPLETED' to detect a return.
What is the mutating-table problem?	A row-level trigger cannot query or change the table that fired it (ORA-04091). That is why the overlap check is inside sp_create_booking.
What does ON DELETE CASCADE do here?	Deleting a booking deletes its payments automatically (Figure 13). Other foreign keys use the default, so a customer with bookings cannot be deleted (ORA-02292).
WHERE vs HAVING?	WHERE filters rows before grouping (role = 'AGENT' in Q15); HAVING filters groups after aggregation (COUNT >= 2).
Why LEFT JOIN in Q2 and Q8?	To keep branches or categories with no matching rows; an inner join would drop the Hitech City branch.
Why is there a COMMIT in each procedure?	Each procedure is one business transaction. If any step fails, RAISE_APPLICATION_ERROR stops it before COMMIT, so a booking is never saved without its advance payment.

8 Conclusion

The Review I design has been implemented in Oracle without changes to the schema. The 8 tables and their 85 constraints protect the data at the lowest level. Six functions compute the derived values (age, rental days, rent, late fee, availability and balance). Five procedures carry out the main business operations, and two cursor procedures produce the overdue and pending-dues reports. Four triggers keep vehicle status, branch and odometer correct without the application having to remember it. The 16 queries cover the reports listed in Review I. For Review III we will connect a Python application to this database so that agents can perform these transactions from a simple interface.